

Python — La visualisation de données

Document 40 — Plotly

Série « Visualisation » · graphiques interactifs et web

Table des matières

- 1. Pourquoi Plotly : le tournant de l'interactif
- 2. Première image : un graphique interactif
 - 2.1 Plotly Express (px) : rapide
 - 2.2 Graph Objects (go) : contrôle fin
- 3. Un graphique complet
- 4. Les types de tracés
- 5. L'interactivité : survol, zoom, légende
- 6. Plusieurs graphiques : facettes et subplots
 - 6.1 Les facettes (facet_col / facet_row)
 - 6.2 make_subplots : composer une figure
- 7. Annotations et repères
- 8. Sauvegarde et export : HTML ou image
- Récapitulatif
- Questions de vérification

1. Pourquoi Plotly : le tournant de l'interactif

Jusqu'ici, la série produisait des images figées (Matplotlib, Pandas, Seaborn). Plotly marque un tournant : sa sortie n'est pas une image, mais une page HTML interactive. L'utilisateur zoome, survole un point pour lire sa valeur, masque une série d'un clic sur la légende. On ne regarde plus un graphique : on l'explore.

C'est le retour à l'étape 4 du pipeline (document 00) : la sortie voulue décide de l'outil. Pour un rapport PDF, Matplotlib ; pour un graphique web ou un tableau de bord où l'on inspecte la donnée, Plotly.

Atout	Ce que ça change
Sortie HTML	Une page web autonome, pas une image — diffusable dans un navigateur.
Interactif	Zoom, survol, info-bulles, légende cliquable, sans code.
Pensé DataFrame	Plotly Express désigne les colonnes par leur nom, comme Seaborn.
Web et dashboards	Base de Dash pour des tableaux de bord complets.

Pourquoi ça compte. Le critère de choix n'est pas « Plotly est-il mieux que Matplotlib » mais « ai-je besoin d'interactivité ». Pour une image dans un document imprimé, Plotly n'apporte rien ; pour explorer ou publier sur le web, il change tout.

2. Première image : un graphique interactif

Comme Matplotlib, Plotly a deux API : une rapide (Plotly Express) et une détaillée (Graph Objects). Les deux construisent le même objet — une Figure Plotly — qu'on affiche avec `fig.show()`.

2.1 Plotly Express (px) : rapide

Plotly Express trace en une ligne depuis un DataFrame, en nommant les colonnes. C'est l'équivalent interactif de Seaborn : on pense en colonnes.

```
import plotly.express as px

df = px.data.stocks()          # jeu de demonstration
fig = px.line(df, x="date", y="GOOG")
fig.show()                    # ouvre dans le navigateur
```

`fig.show()` affiche le graphique interactif : dans un navigateur, ou directement sous une cellule de notebook. Le résultat est déjà zoomable et survolable, sans réglage.

2.2 Graph Objects (go) : contrôle fin

Pour un contrôle précis, on construit la figure trace par trace avec Graph Objects. Plus verbeux, mais on maîtrise chaque détail — utile pour des graphiques composés.

```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Scatter(x=df["date"], y=df["GOOG"],
                        mode="lines", name="GOOG"))
fig.add_trace(go.Scatter(x=df["date"], y=df["AAPL"],
                        mode="lines", name="AAPL"))
fig.show()
```

API	Quand l'utiliser
<code>plotly.express (px)</code>	Tracé rapide depuis un DataFrame ; une fonction par type.
<code>graph_objects (go)</code>	Contrôle fin, figures composées, traces ajoutées une à une.

Pourquoi ça compte. Commencer par Plotly Express : il couvre l'immense majorité des besoins en une ligne. On descend vers Graph Objects seulement quand `px` ne suffit plus — souvent `px` crée la figure, puis on l'ajuste avec les méthodes de `go`.

3. Un graphique complet

Beaucoup de réglages passent en paramètres de la fonction `px`. Pour le reste, on ajuste la figure avec `fig.update_layout` — l'équivalent Plotly des `set_*` de Matplotlib.

```
fig = px.line(
    df,
    x="date", y=["ES", "NQ"],  # plusieurs colonnes -> plusieurs courbes
    title="Comparaison ES / NQ",
    labels={"value": "Prix", "variable": "Actif"},
)
fig.update_layout(
    xaxis_title="Date",
    yaxis_title="Prix de cloture",
)
```

```

        hovermode="x unified",      # une info-bulle commune au survol
        template="plotly_white",
    )
    fig.show()

```

Méthode / paramètre	Rôle
<code>title=</code>	Titre, en paramètre de la fonction <code>px</code> .
<code>labels=</code>	Renomme les axes et la légende (dict colonne -> libellé).
<code>update_layout(...)</code>	Règle titre, axes, légende, marges, thème.
<code>update_traces(...)</code>	Modifie les courbes : couleur, épaisseur, mode.
<code>update_xaxes / yaxes</code>	Réglages fins d'un axe (échelle log, format de date).

Tout est un objet figure. `px` et `go` produisent le même objet `fig`. On le construit vite avec `px`, puis on le peaufine avec les méthodes `update_*`. C'est le même schéma qu'avec Pandas et Seaborn : tracer vite, ajuster ensuite.

4. Les types de tracés

Comme Seaborn, Plotly Express offre une fonction par type de graphique, avec la même signature : le `DataFrame`, puis les colonnes par leur nom.

Fonction	Type	Quand l'utiliser
<code>px.line</code>	Ligne	Évolution dans le temps (prix).
<code>px.scatter</code>	Nuage de points	Relation entre deux variables.
<code>px.bar</code>	Barres	Comparer des catégories (volumes).
<code>px.histogram</code>	Histogramme	Distribution d'une variable (rendements).
<code>px.box</code>	Boîte	Dispersion et valeurs aberrantes.
<code>px.candlestick</code>	Chandelier	OHLC boursier (via <code>graph_objects</code>).

`color=` joue le rôle du `hue` de Seaborn : une couleur par catégorie, et une légende cliquable.

```

fig = px.scatter(df, x="rendement_es", y="rendement_nq",
                 color="regime", size="volume",
                 hover_data=["date"])

fig.show()

```

Pourquoi ça compte. La signature est constante : `px.<type>(df, x=, y=, color=...)`. Comme avec Seaborn, apprendre une fonction, c'est presque les connaître toutes. Le chandelier boursier, lui, passe par `go.Candlestick`.

5. L'interactivité : survol, zoom, légende

L'interactivité est gratuite : chaque figure Plotly est déjà zoomable et survolable. On peut toutefois la régler, et c'est là que Plotly se distingue vraiment des bibliothèques statiques.

Interaction	Comment elle se règle
Survol (hover)	hovertemplate et hover_data : ce qui s'affiche sous le curseur.
Info-bulle unifiée	hovermode="x unified" : une bulle pour toutes les courbes.
Légende cliquable	Active par défaut : un clic masque ou isole une série.
Zoom / pan	Actifs par défaut ; double-clic pour réinitialiser la vue.
Curseur de plage	rangeslider sous l'axe x (séries temporelles).

```
fig = px.line(df, x="date", y="ES")
fig.update_traces(
    hovertemplate="%{x|%d %b}<br>Prix : %{y:.2f} USD",
)
fig.update_xaxes(rangeslider_visible=True) # mini-carte sous l'axe
fig.show()
```

Le hovertemplate. C'est le texte affiché au survol d'un point. On y insère les valeurs avec `%{x}` et `%{y}`, formatées (dates, décimales). C'est ce qui transforme un graphique en outil d'inspection précis.

***Pourquoi ça compte.** L'interactivité n'a de sens qu'à l'écran : elle disparaît dès qu'on exporte en image figée. Si le livrable final est un PNG ou un PDF, autant rester sur Matplotlib. Plotly prend tout son sens quand la page HTML est le livrable.*

6. Plusieurs graphiques : facettes et subplots

6.1 Les facettes (facet_col / facet_row)

Comme le `col=` de Seaborn, Plotly Express éclate un graphique en une grille par catégorie, avec `facet_col` ou `facet_row`. Une ligne suffit.

```
fig = px.line(
    df,
    x="date", y="rendement",
    facet_col="actif",          # un sous-graphique par actif
    facet_col_wrap=2,          # 2 colonnes par ligne
)
fig.show()
```

6.2 make_subplots : composer une figure

Pour des sous-graphiques hétérogènes — un prix au-dessus, un volume en dessous — on construit la grille avec `make_subplots`, puis on ajoute les traces case par case. C'est le pendant interactif de GridSpec (document 10).

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(rows=2, cols=1, shared_xaxes=True,
                    row_heights=[0.7, 0.3])
fig.add_trace(go.Scatter(x=df["date"], y=df["prix"]), row=1, col=1)
fig.add_trace(go.Bar(x=df["date"], y=df["volume"]), row=2, col=1)
fig.show()
```

Pourquoi ça compte. `facet_col` / `facet_row` pour éclater une même mesure par catégorie (en une ligne avec `px`) ; `make_subplots` pour composer des graphiques de natures différentes (prix + volume). C'est exactement la distinction `subplots` / `GridSpec` du document 10, version interactive.

7. Annotations et repères

On enrichit une figure Plotly avec du texte et des repères, comme avec Matplotlib, mais via des méthodes dédiées de l'objet figure.

Méthode	Rôle
<code>add_annotation</code>	Texte (avec flèche optionnelle) à une position donnée.
<code>add_hline</code> / <code>add_vline</code>	Ligne horizontale / verticale de repère (seuil, date).
<code>add_hrect</code> / <code>add_vrect</code>	Zone colorée entre deux valeurs (plage de séance).

```
fig = px.line(df, x="date", y="prix")
fig.add_hline(y=100, line_dash="dash", line_color="red",
              annotation_text="Support")
fig.add_annotation(x="2024-03-01", y=120,
                  text="Plus haut", showarrow=True)
fig.show()
```

Des repères qui restent interactifs. Une ligne ajoutée avec `add_hline` se déplace et se zoome avec le reste du graphique. Le repère fait partie de la figure, pas d'une image plaquée par-dessus.

8. Sauvegarde et export : HTML ou image

C'est le choix décisif avec Plotly, et il découle directement de l'étape 4 du pipeline : veut-on conserver l'interactivité, ou produire une image figée ?

Export interactif (HTML). `write_html` écrit une page web autonome : on garde le zoom, le survol, la légende cliquable. C'est l'usage naturel de Plotly.

```
fig.write_html("rapport.html") # page interactive autonome
```

Export statique (image). `write_image` produit un PNG, un PDF ou un SVG figé — pratique pour glisser le graphique dans un document. Nécessite le moteur kaleido.

```
# pip install kaleido
fig.write_image("rapport.png", scale=2) # image figée, haute résolution
fig.write_image("rapport.pdf")         # vectoriel
```

Méthode	Sortie
<code>fig.show()</code>	Affichage interactif (navigateur ou notebook).
<code>write_html</code>	Page HTML autonome, interactive : l'usage propre à Plotly.
<code>write_image</code>	Image figée (PNG/PDF/SVG) ; perd l'interactivité ; via kaleido.
<code>write_json</code>	Séréalise la figure pour la recharger ou la passer à Dash.

Pourquoi ça compte. Exporter en HTML si l'interactivité est l'intérêt (le destinataire explore) ; en image seulement pour un document figé — mais dans ce cas, Matplotlib aurait souvent suffi. Pour aller plus loin que le graphique isolé, Dash (de l'éditeur de Plotly) assemble plusieurs figures en un tableau de bord web.

Récapitulatif

- 1. Le tournant.** Plotly produit du HTML interactif, pas une image : on explore (zoom, survol, légende cliquable).
- 2. Deux API.** Plotly Express (px, rapide, depuis un DataFrame) et Graph Objects (go, contrôle fin). Même objet figure.
- 3. Un graphique complet.** Paramètres de px, puis `fig.update_layout / update_traces` pour titre, axes, thème.
- 4. Les types.** `px.line/scatter/bar/histogram/box...` signature constante ; `color=` joue le rôle de `hue`.
- 5. L'interactivité.** `hovertemplate`, `hovermode`, légende cliquable, `rangeslider` — gratuite, mais réglable.
- 6. Plusieurs graphiques.** `facet_col/row` (px, une mesure par catégorie) et `make_subplots` (compositions hétérogènes).
- 7. Repères.** `add_annotation`, `add_hline/vline`, `add_hrect/vrect` — interactifs avec la figure.
- 8. Export.** `write_html` (interactif, l'usage propre) vs `write_image` (figé, via kaleido).

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Q1. Qu'est-ce qui distingue fondamentalement Plotly de Matplotlib, Pandas et Seaborn ?

Réponse. Sa sortie : Plotly produit une page HTML interactive (zoom, survol, légende cliquable), là où les trois autres produisent une image figée. C'est l'outil de l'exploration et du web, pas du document imprimé.

Q2. Quelles sont les deux API de Plotly, et laquelle privilégier au début ?

Réponse. Plotly Express (px), rapide, qui trace en une ligne depuis un DataFrame ; et Graph Objects (go), détaillé, qui compose la figure trace par trace. On commence par px, et on descend vers go seulement quand px ne suffit plus.

Q3. Comment règle-t-on le titre, les axes et le thème d'une figure Plotly ?

Réponse. Le titre (et labels) en paramètres de la fonction px, puis le reste avec `fig.update_layout(...)` — l'équivalent des `set_*` de Matplotlib. `update_traces` ajuste les courbes elles-mêmes.

Q4. À quoi sert le paramètre `color=` dans Plotly Express ?

Réponse. À associer une couleur à chaque valeur d'une colonne catégorielle, avec une légende cliquable. C'est l'exact équivalent du `hue=` de Seaborn : il encode une variable supplémentaire.

Q5. Quelle est la différence entre `facet_col` et `make_subplots` ?

Réponse. `facet_col` (ou `facet_row`) éclate une même mesure en une grille par catégorie, en une ligne avec px. `make_subplots` compose une figure de graphiques hétérogènes (prix + volume), trace par trace. C'est la distinction `subplots / GridSpec` du document 10, version interactive.

Q6. Quels sont les deux modes d'export, et quand choisir chacun ?

Réponse. `write_html` produit une page web autonome qui conserve l'interactivité — l'usage naturel de Plotly. `write_image` produit une image figée (PNG/PDF/SVG, via kaleido) pour un document, mais dans ce cas Matplotlib aurait souvent suffi.

Document suivant (50) : *PyQtGraph — visualisation temps réel haute performance : flux de marché en direct, rafraîchissement rapide, basse latence, là où Matplotlib peine.*